TK499 storage space and Bootloader written

1、 **TK499 memory allocation**

TK499 memory address from 0x0x70000000 for a total of 8MB; At system startup, the ROM solidified inside the chip and the official default Bootloader occupy a total of the first 128KB at startup, usually 128KB It is not used by default, but after starting into the main program, the user can still reuse it. The official default bootloader divides the remaining memory into two chunks, followed by the opening 128KB to store the code, currently allocating 2MB of capacity, and the rest (8MB-128KB-2MB) as running memory for the application. BecauseBootloader users can write their own, users can allocate 1MB as storage code, leaving the remaining 6.875MB (8 MB-128KB-1MB) as the application's running memory is also OK. Or any other allocation method.

2、 **TK499 boots flash and memory actions**

At the start of the chip, the ROM cured inside the chip is responsible for starting the DMA, the QSPI module, reading the external FLASH (e.g. W28Q128), if there is a program, directly to the user boot The memory address defined by the loader is stored on it, and after moving, it jumps directly to the memory to run. If there is no program in FLASH, then the chip will start the USB module again, and enter the download mode at the same time, the user can use the USB to download the bootloader made by himself. You can also download the application directly (KEIL set FLASH address to 0x70008000 and TK499.) without making bootloader h #define T_SDRAM_BASE 0x70008000).

ROM is to identify the program description information starting from the 0 address of the external SPI FLASH, and then carry the corresponding length code to the address of the memory 0x70008000 as instructed by the information. Because ROM is made in the metal layer of the chip, it is very slow, 48MHz clock, if you put a very large program, such as 2MB, ROM will be very slow to carry. For faster speed, the bootloader product appeared. First let ROM move a dozen KB Bootloader into memory, and then run Bootloader to three or four times the speed of ROM, and carry large programs into memory to run. So if your program is not large, or the startup speed is not too high, you can not want Bootloader Of course, bootloader can make richer functions, such as remote upgrades.

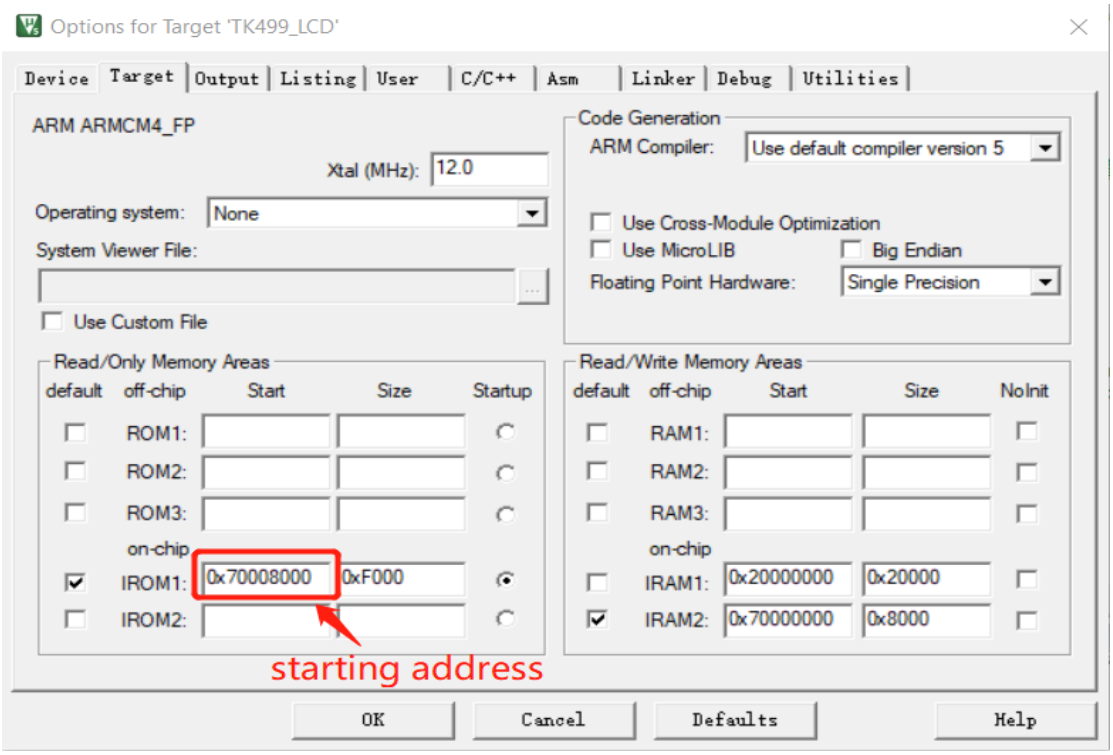
3、 **Key information produced by Bootloader**

Make a bootloader, mainly to understand the information description and key addresses of the program header. Mainly involves four addresses, ①the bootloader and the application program APP respectively exist in the SPI FLASH which address; ②The bootloader and application APP are respectively moved to which address of memory (that is, the running address in SDRAM).

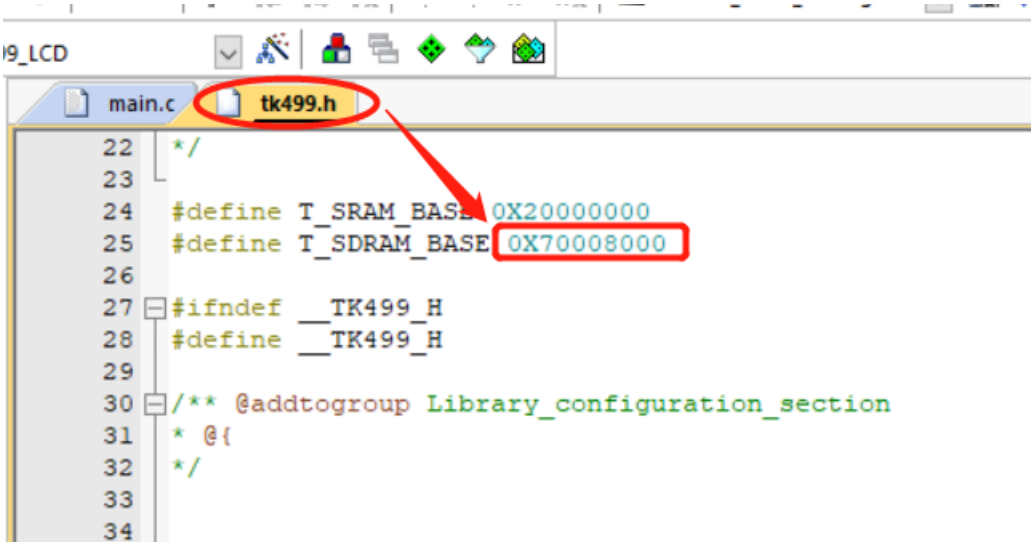
First, because ROM is solidified inside the chip, ROM fixes the first running program (bootloader) from SPI FLASH Starting with the 0 address, it is also fixed that the first running address of the bootloader in SDRAM is 0X70008000.

When ROM is running, the first 24 bytes of the 0 address of SPI FLASH are first read via Q SPI; The first 32 bits (4 bytes) interpret the start address of the

program stored in RAM, which is consistent with the KEIL setting, and the second 32 bits is the starting address The third 32 bits is the length of the program, and the fourth is the inverse code of length; The fifth 32 bit is the constant 0x12346789, and the sixth is the inverse code of the constant 0x12346789; After ROM explains this information, it checks the front and back codes, if it confirms the match, it means that there is a program, and if it finds a mismatch, it means that there is no program, then it has to enter the download mode. When a match is detected, ROM will carry the specified length and start address from the offset address of (0 address + 1KB) to the starting address specified in SDRAM, and move the jump run. The starting address of the bootloader in the current sample is 0X70008000, as shown in the following figure:



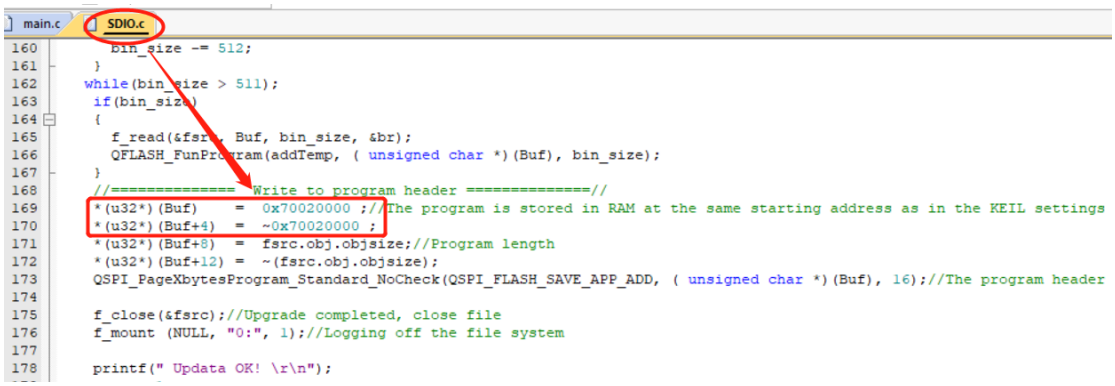
With the program, TK499.h in the same address:



Then, in the bootloader, you can freely define the starting address of your application in SPI FLASH, and the address of the program to run, to move to memory; (Note: This define address cannot conflict with the address space in front of it).)

Example bootloader defines the user application description information at the 0 X10000 address of the SPI FLASH, the same offset 1KB is the program body, see "SDIO.C" Line 110 comment;

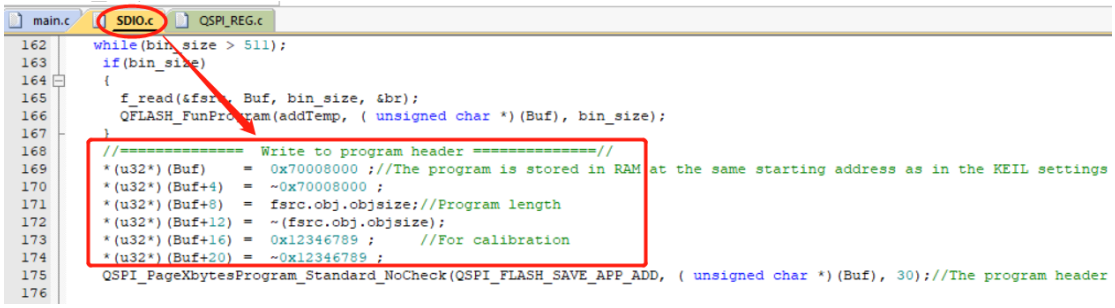
Finally, define the address that the user application holds in SDRAM



The Bootloader is written here, remember that your application will be set according to the start address defined by your Bootloader. The KEIL settings must be the same as those preset by your Bootloader to run.

4、 **The way to write directly to the main program** without a boo**tloader**

Programs that rely on bootloader, first burn bootloader to burn the main program, in some cases it is still a bit troublesome, but the main program does not have a bootloader It can also be run. In fact, bootloader is a main program, but it is a main program dedicated to downloading programs. The existence of Bootloader is only that the chip starts to carry the program to the internal SDRAM faster, only at startup, and the bootloader also takes up 64K space. There is no need to rely on the bootloader, and the algorithm of the direct main program is basically the same as that of the bootloader. First of all, the program is stored on FLASH starting from the 0 address, and secondly, these lines of code have differences:



TKM32F499 Official Bootloader Description

If there is no special need, you can also use the official Bootloader The official Bootloader runs at 192M, only supports USB download mode, supports downloading

BIN programs, TXT, and so on Files in several formats, such as JPG, GIF, XBF, and HJR.

When the chip is first used, there is no program, only the ROM that is cured inside the chip can be executed. When the chip starts, the R OM is run first. ROM automatically performs chip initialization, starts the relevant clock, starts QSPI, DMA, UART and USB, so the first time the chip is run, it has already been Support USB download mode. ROM then detects the 0 address program description information of the external S PI FLASH (e.g. W25Q128) and if it finds that there is no relevant information, That jumps directly to download mode. The Bootloader will download the program to 0 X10000+ 0X400 in the external F LASH, and then after downloading the program, it will write the address and program length of the program running in the DRAM to 0 X10000. The Bootloader reserves 2MB of space by default for the stored program, so the address after 0X210000 in S PI FLASH is used to store data. If you download the image library data, bootloader will store the relevant data in the order after 0X210000. If you want to get your data header address, you can use the get_file_address_NOR_FLASH function to get it. In the above download program, data, etc., there are related important information will be printed out through serial port 1, you can use 460800 Baud rate to go to the monitor.

Note: Because the current version of Bootloader does not know what capacity of FLASH you plug in, it is not clear whether the microcontroller is full of downloads, and the user needs to stop throwing files into it. You can also use serial ports to monitor. Or you can first select all the files to be thrown in, and then throw them in if they are not oversized on the computer. In addition, if you lose too much, or lose duplicates, you can throw this file "DEL56789.txt" into it and empty the data. Or if you re-download the Bootloader, the user data will also be deleted.

All the files thrown in at a time cannot exceed 7.8MB, because the pop-up USB stick is virtual with the memory of the microcontroller. A total of 8MB of memory, ROM and Bootloader occupy 128K, the file system is also used a little, the largest single file can only reach 7. 8MB, if your total file size exceeds 7.8MB, please throw it in multiple times. This Bootloader storage file function only supports 16MB at most, and then it is too big to study it yourself, to identify the file is greater than 16MB, expand the address to 4 bytes to do it, this official BootLoader is not that complicated.

The documents to be used by the official Bootloader are in the "TKM32F499 Evaluation Board" data kit, of which "DEL56789.txt" is in the "Chinese-English Font Library, Digital Tube Font .rar" One of the get_file_address_NOR_FLASH function application examples in routine 16 "16. TK499 LCD TK035F5589 FLASH Font picture auto get address.zip". In addition, regarding FLASH reads and writes, the relevant functions are in the qspi_fun.c and “QSPI_REG.c”, a total of two sets of libraries, of which the qspi_fun.c is mainly D MA is written, QSPI_REG.c focuses on polling.

To run Bootloader, you only need the smallest system on a chip. Mainly including 12M crystal oscillator, USB interface, SPI FLASH (capacity recommended W25Q16 or more), download button (PA1 and PA13, button common solar) single 3.3V power supply.